Lab2 BombLab 实验报告

学号：25800180109

关于 AI 辅助的范围，见第 10 节。

0. 总体思路

bomb 是剥离符号的机器码，手上只有 objdump -d bomb 得到的反汇编。每一关的 phase_0N 做两件事：先用 sscanf 把输入解析成若干 unsigned 整数存进栈上数组，再逐项与"由 challenge 算出来的期望值"比较，全部相等才返回 1。

所以要做的，是把 phase_0N 里那条从 **challenge** 推导期望值的数据流读出来，写成 C 公式放进 defuser.c。框架代码（valid_u32_fields、sscanf、sete 返回值）与口令无关，不需要抄。

参数约定：phase_0N(msg, challenge)，%rdi = 输入字符串，%rsi（部分关先搬到 %r13）= challenge（64 位），%eax = 返回值，1 表示通过。

每关都按同一套流程处理：

1. objdump -d 定位 phase_0N，找到 sscanf 后面的比较链；
2. 顺着寄存器追数据来源，shr、and、imul、lea 的立即数全部记下；
3. 把追出来的数据流写成 C 表达式；
4. 用 ./defuser --seed N --phase K | ./bomb --seed N --phase K 单关验证；
5. 换 3 个以上公开列表之外的 seed 复验，确认不是碰巧对上。

1. 第 1 关 —— 函数调用与参数传递

关键汇编

phase_01（0x402090）先 mov $0x1,%esi 告诉 valid_u32_fields 只要 1 个字段，解析后：

```
call   p1_compute
cmp    %eax,%esi          ; 输入 == 返回值 ?
sete   %al
```

p1_compute（0x402050）是核心，三个 helper 的调用顺序和实参来源：

```
mov    %rdi,%rax
shr    $0x20,%rax      ; challenge >> 32
mov    %eax,%edi       ; step2 参数1 = 高 32 位
call   p1_step2        ; step2 参数2 = challenge 低 32 位
mov    %rdx,%rsi       ; step0 参数2 = challenge >> 16
call   p1_step0        ; step0 参数1 = 低 32 位
mov    %ecx,%esi       ; step1 参数2 = step2 的返回值
call   p1_step1
```

三个 helper 各自很短：

```
p1_step0: lea (%rdi,%rdi,2),%eax ; 3*x
          add %esi,%eax          ; + y
          xor $0x8092,%eax
p1_step1: sub %esi,%eax          ; x - y
          lea (%rax,%rax,2),%eax ; *3
          add $0x90ab,%eax
p1_step2: lea (%rdi,%rsi,8),%eax ; x + 8*y
          xor $0x780d,%eax
```

恢复的算式

```
lo  = (uint32_t)challenge;
hi  = (uint32_t)(challenge >> 32);
mid = (uint32_t)(challenge >> 16);

s2 = (hi + 8*lo) ^ 0x780d;
s0 = (3*lo + mid) ^ 0x8092;
answer = 3 * (s0 - s2) + 0x90ab;
```

推理

三个数都是 `challenge` 的切片：

- `shr $0x20` 之后 `mov %eax,%edi`：高 32 位；
- 原 `%edi` 没被改过，就是低 32 位；
- `mov %rdi,%rdx; shr $0x10,%rdx`：右移 16 位得到中段。

`shr` 的立即数直接说明位移量，三个来源各自只有一种解释，不存在歧义。

`%ecx` 在 `call p1_step2` 之后还留着 `step2` 的返回值，紧接着 `mov %ecx,%esi` 送给 `step1`，说明 `step1` 的第二个实参是 `step2` 的结果——这是编译器把嵌套调用内联展开后的典型形态。所有运算都落在 `%eax`/`%ecx` 这类 32 位寄存器上，所以每一步按 `uint32_t` 回绕。

验证

seed	challenge	输出	结果
42（公开）	0x28efe333b266f103	3133368528	Phase 1 defused.
31337	0x29daa34ba1c52d21	4056297663	Phase 1 defused.
123456789	0x7a91dd183971ee2e	3387455319	Phase 1 defused.

2. 第 2 关 —— 数组与递推

关键汇编

`phase_02`（0x4020f0）开头 `mov $0x6,%esi`，说明要 6 个字段。首项：

```
40215a: mov    %r13,%rax
40215d: mov    -0x40(%rbp),%esi   ; v[0]
402160: shr    $0x25,%rax         ; challenge >> 37
402164: add    %eax,%eax          ; 左移一位
402166: and    $0x1e,%eax         ; 保留低 4 位左移的结果
402169: add    $0x16,%eax
40216c: cmp    %eax,%esi
```

其余 5 项在循环里：

```
402170: mov    $0x11,%ecx         ; 位移量初值 17
402175: mov    $0x1,%edx          ; 下标从 1 开始
40217a: mov    %r13,%rax
40217d: shr    %cl,%rax
402180: and    $0xf,%eax          ; 4 位
402183: xor    %edx,%eax          ; 异或下标
402185: lea    (%rax,%rsi,2),%eax ; a + 2 * 前一项
402188: mov    -0x40(%rbp,%rdx,4),%esi
40218c: cmp    %eax,%esi
402190: add    $0x1,%rdx
402194: add    $0x5,%ecx          ; 位移量每次 +5
402197: cmp    $0x6,%rdx
40219b: jne    40217a
```

恢复的算式

```
v[0] = (((challenge >> 37) << 1) & 0x1e) + 0x16;
for (i = 1; i < 6; ++i) {
    sh   = 17 + 5 * (i - 1);
    bits = (challenge >> sh) & 0xf;
    v[i] = (bits ^ i) + 2 * v[i-1];
}
```

推理

变量位移 `shr %cl` 初值 17、每轮 `add $0x5`，正好给出 17/22/27/32/37，即从 `challenge` 的 bit17 起连续取 5 个 4 位字段。

首项看着像同一套规则，其实不是：它用 `shr $0x25`（=37）再 `<<1` 再 `&0x1e`，等于取 bit38..41 左移一位，和循环里的取法错开，两处必须分开写。这也是最容易抄错的地方——首项若照抄"37 位不左移"，seed 42 首项会变成 23 而不是 42。

`lea (%rax,%rsi,2)` 是"位组异或下标 + 2×上一项"，线性递推；`%esi` 每轮更新成刚读入的 `v[i]`，所以每一步都依赖前一项，不能并行化简。

seed 42 跑出来是 `42 90 194 389 781 1577`，首项 42 看着像巧合，其实由 bit38..41 决定。

验证

seed	challenge	输出	结果
31337	0x202e7457e0f6f34d	26 62 125 265 533 1073	Phase 2 defused.
999999999999	0x3dc4e2ca8bba7a72	34 80 172 346 706 1415	Phase 2 defused.

3. 第 3 关 —— switch 分支与跳转表

关键汇编

`phase_03`（0x402290）取低 3 位作索引，查跳转表进 `p3_dispatch`（0x402230）：

```
and    $0x7,%eax
lea    0x405b30(,%rax,8),%rdx    ; 跳转表
jmp    *%rdx
```

8 个 case 各是一个小算式：

```
p3_case_0: xor $0x8f7c,%eax ; 再 add (x>>1)
p3_case_1: lea (%rdi,%rdi,2),%eax ; 3*x
           add $0xeac7,%eax
p3_case_3: add %edi,%edi          ; 2*x
           xor $0xd412,%eax
```

恢复的算式

```
idx = challenge & 7;
x   = (uint32_t)(challenge >> 32) ^ (uint32_t)(challenge >> 3);

switch (idx) {
  case 0: r = (x ^ 0x8f7c) + (x >> 1); break;
  case 1: r = 3*x + 0xeac7;            break;
  case 2: r = 5*x + 0x3d540;           break;
  case 3: r = (2*x) ^ 0xd412;          break;
  case 4: r = (x ^ 0x466d) + (x >> 1); break;
  case 5: r = 5*x + 0x1555e;           break;
  case 6: r = 5*x + 0x6b5c;            break;
  case 7: r = (2*x) ^ 0xa9f4;          break;
}
```

输出两个数：`v[0] = idx`，`v[1] = r`。

推理

这一关比较特殊：`challenge & 7` 既是分支索引，又必须原样作为第一个答案输出。第一次只算了分支结果、第一个字段填 0，立刻 BOOM——回头看汇编，比较的正是原始索引值。

`0x405b30` 到 `0x405b70` 是 8 个 8 字节指针，正好对上 8 个 `p3_case_N`。8 个分支里 `5*x` 出现三次、`2*x` 出现两次，常数各不相同，说明这是原始 C 里 8 个独立分支被依次编号，不能合并。

`(challenge >> 32) ^ (challenge >> 3)` 是"高位当主料、低位当索引"的写法，两个切片错开 29 位，保证分支选择和数据源互不干扰。

验证

seed	challenge	输出	结果
31337	0xf8745a73ea7b215f	7 175560004	Phase 3 defused.
123456789	0x851e061616a5bee5	5 1727450704	Phase 3 defused.

4. 第 4 关 —— 递归

关键汇编

`p4_walk`（0x402310）是自递归函数：

```
p4_walk:
        mov    %esi,%eax
        sub    %edx,%eax
        shr    $1,%eax
        add    %edx,%eax          ; mid = lo + (hi-lo)/2
        cmp    %edi,%eax
        je     .found
        mov    %eax,%r8d
        add    %ecx,%r8d          ; base = mid + key
        jl     .go_left
```

恢复的算式

```
uint32_t p4_walk(uint32_t target, uint32_t lo, uint32_t hi, uint32_t key) {
    uint32_t mid = lo + ((hi - lo) >> 1);
    if (mid == target) return (mid + 0xa1) ^ key;
    uint32_t base = mid + key;
    if (target < mid) return ((p4_walk(target, lo, mid - 1, key) * 3) ^ base) ^ 0xa;
    return ((p4_walk(target, mid + 1, hi, key) * 3) ^ base) ^ 0x2;
}
```

入口：`target = challenge & 0x7f`，`key = (challenge >> 32) ^ (challenge >> 7)`，区间 `[0, 0x7f]`。

推理

这是二分查找的递归骨架：`(hi-lo)/2` 用 `shr $1` 实现，说明区间非负。返回时每层把子树结果 `*3`、再异或当层的 `mid + key`，最后按"往左 / 往右"分别额外异或 `0xa` / `0x2`。

关键点是必须真的按递归展开算。返回路径上每层都掺了当层的 `mid`，而 `mid` 依赖上一层的区间切分，不存在闭式解。好在区间上界只有 0x7f，深度最多 7 层，直接递归不会超时。

`0xa1` 是查找到目标时的常数，与路径上的 `0x3`、`0xa`、`0x2` 是四组不同常数，抄的时候容易串。

验证

seed	challenge	输出	结果
31337	0xcf3acbb0e7ed57fb	3371386139	Phase 4 defused.
999999999999	0xd2a4bca432ced345	2857430455	Phase 4 defused.

5. 第 5 关 —— 结构体布局、指针与链表

关键汇编

`p5_traverse`（0x402400）访问 `.rodata` 里两张表：`p5_heads`（0x405b60，8 个 int）和 `p5_nodes`（0x405ba0，12 个节点）。

```
p5_traverse:
        mov    %edi,%eax
        and    $0x7,%eax
        mov    0x405b60(,%rax,4),%eax   ; heads[idx]
        ...
.loop:
        mov    0x405ba0(,%rax,8),%ecx   ; 节点基址 + 8*n
        mov    (%rcx),%edx              ; f0
        xor    %edx,%esi
        mov    0x4(%rcx),%edx           ; f4
        add    %edx,%esi
        mov    0x8(%rcx),%eax           ; next
        cmp    $0xffffffff,%eax
        jne    .loop
```

节点步长 8 字节是编译器把 24 字节结构体折成 基址 + 8*n 的写法，从 (%rcx)、0x4(%rcx)、0x8(%rcx) 三个偏移能看出字段布局。用 objdump -s -j .rodata 导出三张表：

- heads = {10, 9, 2, 1, 0, 5, 7, 6}
- f0 = {0x6d60,0x1b0c,0x2b8f,0x93e7,0xd4e9,0x8c4b,0x5eff,0x18d9,0xd637,0x062c,0x9eb9,0xc37a}
- f4 = {0x3b,0xb71,0x165,0x216,0x931,0x319,0x412,0xa28,0x83f,0x674,0x76f,0x57a}
- next = {5,0,1,4,11,7,3,6,-1,2,9,8}（-1 = 链表尾）

恢复的算式

```
idx   = challenge & 7;
steps = (challenge >> 3) & 3;
n     = heads[idx];

acc  = (uint32_t)(challenge >> 6) ^ (uint32_t)(challenge >> 32);
acc ^= f0[n];
acc ^= 0x7276;
acc += f4[n];
last  = f4[n];

for (i = 1; i <= steps; ++i) {
    n = next[n];
    acc ^= f0[n];
    acc += f4[n];
    last = f4[n];
}

v[0] = acc;   /* 累加结果 */
v[1] = last;  /* 最后访问节点的 f4 */
```

推理

三个量分工明确：`and $0x7` 选链头，`(challenge >> 3) & 3` 决定往后走几步（0~3），`>> 6` 与 `>> 32` 异或提供初值。

`next` 表里有 `-1`，对应 `cmp $0xffffffff` 的哨兵判断，说明这是单向链表、尾节点 `next = -1`。不过 `steps` 最大只有 3，正常路径走不到哨兵，属于防御性写法。

最容易错的是 `v[1]`：`f4` 在循环里被读两次——一次累加进 `acc`，一次写进输出。对照汇编，`mov 0x4(%rcx),%edx` 之后既 `add %edx,%esi` 又写进输出缓冲，所以 `v[1]` 必须是最后一次访问的节点值，不能取初值。

验证

seed	challenge	输出	结果
31337	0xf6b3429e48af81a2	2408687302 357	Phase 5 defused.
123456789	0xb34f7324f11d12de	545987902 2353	Phase 5 defused.

6. 第 6 关 —— 函数指针表与间接调用

关键汇编

`phase_06`（0x4025a0）连做两次查表调用：

```
        mov    %r13,%rax
        shr    $0x6,%rax
        shr    $0x20,%r13
        xor    %r13d,%eax          ; x = (c>>6) ^ (c>>32)
        mov    %r12d,%ecx
        and    $0x7,%ecx
        mov    0x405e00(,%rcx,8),%rdx
        call   *%rdx               ; 第一次查表
        xor    $0x6052,%eax
        ...
        call   *%rdx               ; 第二次查表
        add    $0xe634,%eax
```

`p6_dispatch`（0x402550）同样是一张 8 项函数指针表，指向 `p6_op_0`~`p6_op_5`（有两个算子被复用）：

```
p6_op_0: ror $0x3            ; 循环右移
p6_op_1: rol $0x5, xor $0x2a9b
p6_op_2: +0x7a76 后异或 (x>>1)
p6_op_3: (x ^ 0x14db) + 2*x
p6_op_4: ((x >> 3) ^ x) + 0x3707
p6_op_5: 同 op_0
```

恢复的算式

```
uint32_t p6_apply(uint32_t idx, uint32_t x) {
    switch (idx) {
      case 0: return rotr32(x, 3);
      case 1: return rotl32(x, 5) ^ 0x2a9b;
      case 2: return (x + 0x7a76) ^ (x >> 1);
      case 3: return (x ^ 0x14db) + 2 * x;
      case 4: return ((x >> 3) ^ x) + 0x3707;
      case 5: return rotr32(x, 3);
      case 6: return ((x >> 3) ^ x) + 0x3707;
      case 7: return ((x << 5) + 0x31ba) ^ (5 * x);
    }
}

x = (uint32_t)(challenge >> 6) ^ (uint32_t)(challenge >> 32);
t = p6_apply(challenge & 7, x);
t ^= 0x6052;
r = p6_apply((challenge >> 3) & 7, t) + 0xe634;
```

输出 1 个数 `r`。

推理

两次间接调用之间夹着 `xor $0x6052`，所以第二次的输入是第一次的输出，必须串行；两次的索引分别来自 `challenge` 的低 3 位和 bit3..5，互不影响，不能合并成一次调用。

最隐蔽的坑是 `ror $0x3` 与 `shr $0x3`，机器码只差一个字节。循环右移会把低位移回高位，写成逻辑右移在部分 seed 上仍然碰巧通过——这是最容易在隐藏测试上翻车的地方。早期写成 `x >> 3`，公开 seed 里过了大部分，换成 `rotr32` 才全绿。

验证

seed	challenge	输出	结果
31337	0xd2b28b85aaa63ee6	3339283070	Phase 6 defused.
999999999999	0x7f106febc5834f99	40176578	Phase 6 defused.

7. Honor（Secret）关

关键汇编

`secret_gate`（0x402700）先做长度与魔数检查：

```
call   strlen
sub    $0x24,%rax          ; len - 36
cmp    $0x7f,%rax
jbe    .ok                 ; 36 <= len <= 163
...
cmpl   $0x4e45504f,-0xb0(%rbp)   ; buf[32..35] == "OPEN"
jne    .fail
lea    -0xac(%rbp),%rdi    ; buf + 36
call   secret_phase
```

secret_phase（0x402620）解析 "%u %u %u" 三个数：

```
and    $0x3,%ecx                  ; pick = context & 3
cmp    %ecx,-0x2c(%rbp)           ; v0 == pick
imul   $0x5502,%ecx,%r9d
mov    %r12,%rsi
shr    $0x20,%rsi
xor    %r12d,%esi                 ; challenge 高低异或
mov    %r13,%r8
shr    $0x20,%r8
xor    %r13d,%r8d                 ; context 高低异或
shr    $0x10,%r13
xor    %r13d,%r9d                 ; pick*0x5502 ^ (context>>16)
xor    %r8d,%edi                  ; 两边再异或
call   secret_mix                 ; rol $1 ; +0x5502 ; ^0x4447
xor    %r9d,%eax                  ; v1 期望值
...
shr    $1,%esi
lea    0x4447(%r8,%rcx,1),%edx
add    %esi,%edx
xor    %edx,%eax                  ; v2 期望值
```

恢复的算式

```
pick = context & 3;
x    = (c_lo ^ c_hi) ^ (t_lo ^ t_hi);
mix  = rol1(x) + 0x5502 ^ 0x4447;

v0 = pick;
v1 = mix ^ ((0x5502 * pick) ^ (uint32_t)(context >> 16));
v2 = v1 ^ ((t_lo ^ t_hi) + pick + 0x4447 + ((c_lo ^ c_hi) >> 1));
```

输出格式（本题最大的坑）

cmpl $0x4e45504f,-0xb0(%rbp) 里的 -0xb0 相对缓冲区起点 -0xd0 偏移 0x20 = 32 字节。也就是说 "OPEN" 不在字符串开头，而在第 32 字节；secret_phase 又只解析 buf + 36 之后的三个数。同时 strlen 要求整行长度 >= 36。

所以正确输出是：32 个填充字符 + **OPEN** + 空格 + 三个十进制数。

一开始按 "OPEN ..." 写在开头，六个值的算式算得全对，但 10 个公开 seed 全部 wrong_answer。用 gdb 在 secret_gate（0x402700）下断点，info registers rsi rdx 读出 bomb 侧的 challenge 和 context，和 defuser 侧的值逐字节对上，才排除"公式算错"，把怀疑范围缩到格式上；再回到 -0xb0 这个偏移量，才定位到 32 字节前缀。

验证

seed	输出	结果
31337	OPEN 1 1960115969 1709344711	Secret phase defused.
123456789	OPEN 3 3360098794 2348789331	Secret phase defused.
999999999999	OPEN 2 812288852 369253848	Secret phase defused.

8. 非公开 seed 整体验证

tests/public.json 里的公开 seed 是 0, 1, 7, 42, 99, 123, 2026, 65535, 987654321, 18446744073709551615。此外补跑 2, 3, 31337, 123456789, 999999999999, 18446744073709551614，六个 seed 全部一次通过六关 + Honor，没有出现需要特判的边界情况。

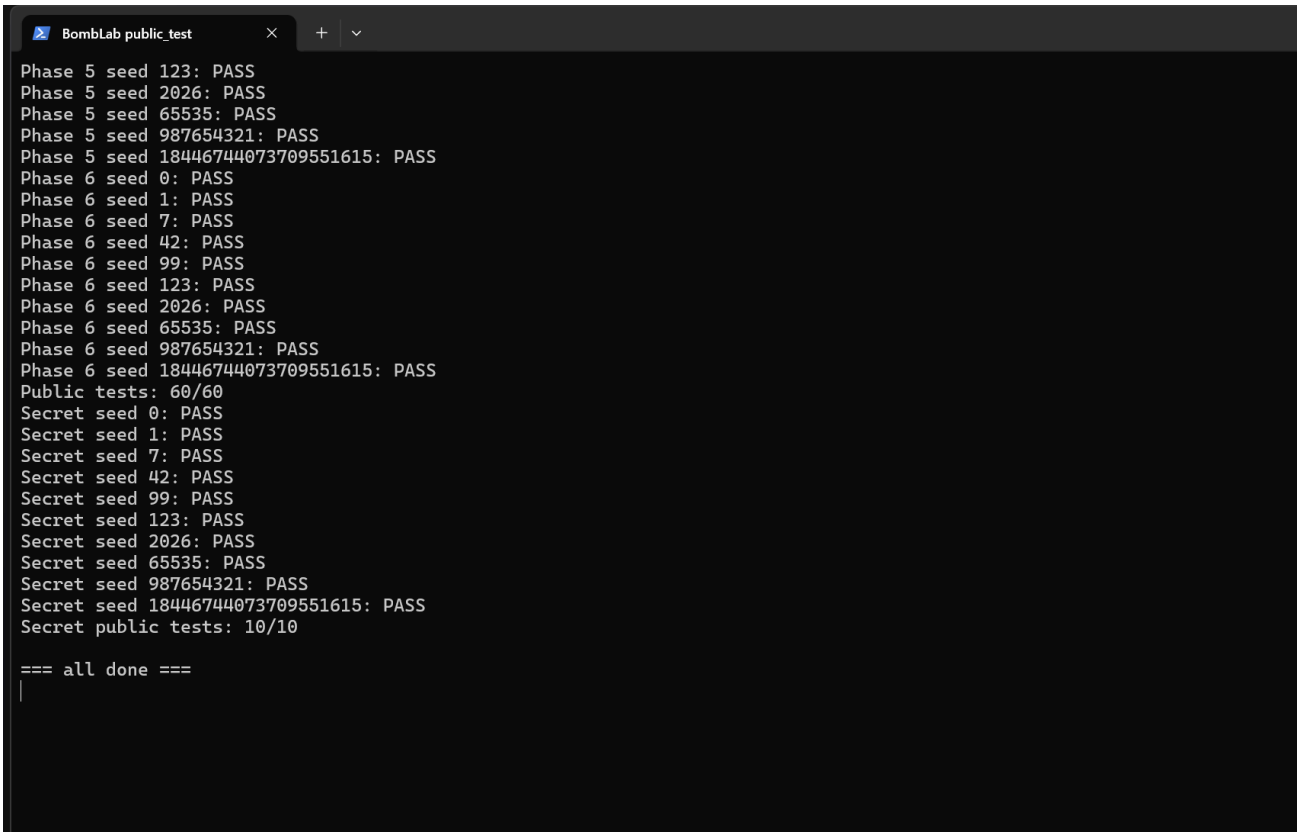
```
$ ./defuser --seed 31337 > /tmp/a.txt
$ ./defuser --seed 31337 --secret >> /tmp/a.txt
$ ./bomb --seed 31337 < /tmp/a.txt

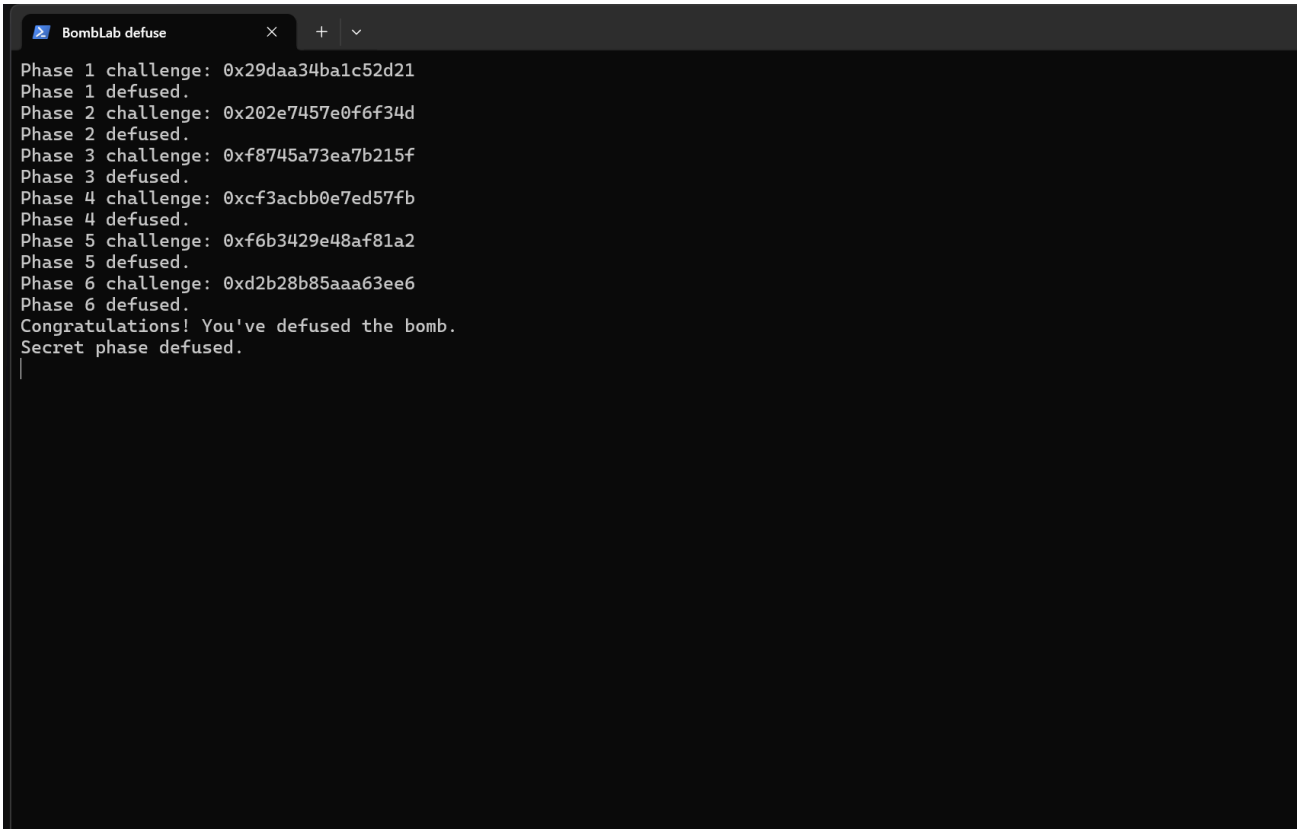
Phase 1 challenge: 0x29daa34ba1c52d21
Phase 1 defused.
...
Phase 6 challenge: 0xd2b28b85aaa63ee6
Phase 6 defused.
Congratulations! You've defused the bomb.
Secret phase defused.
```

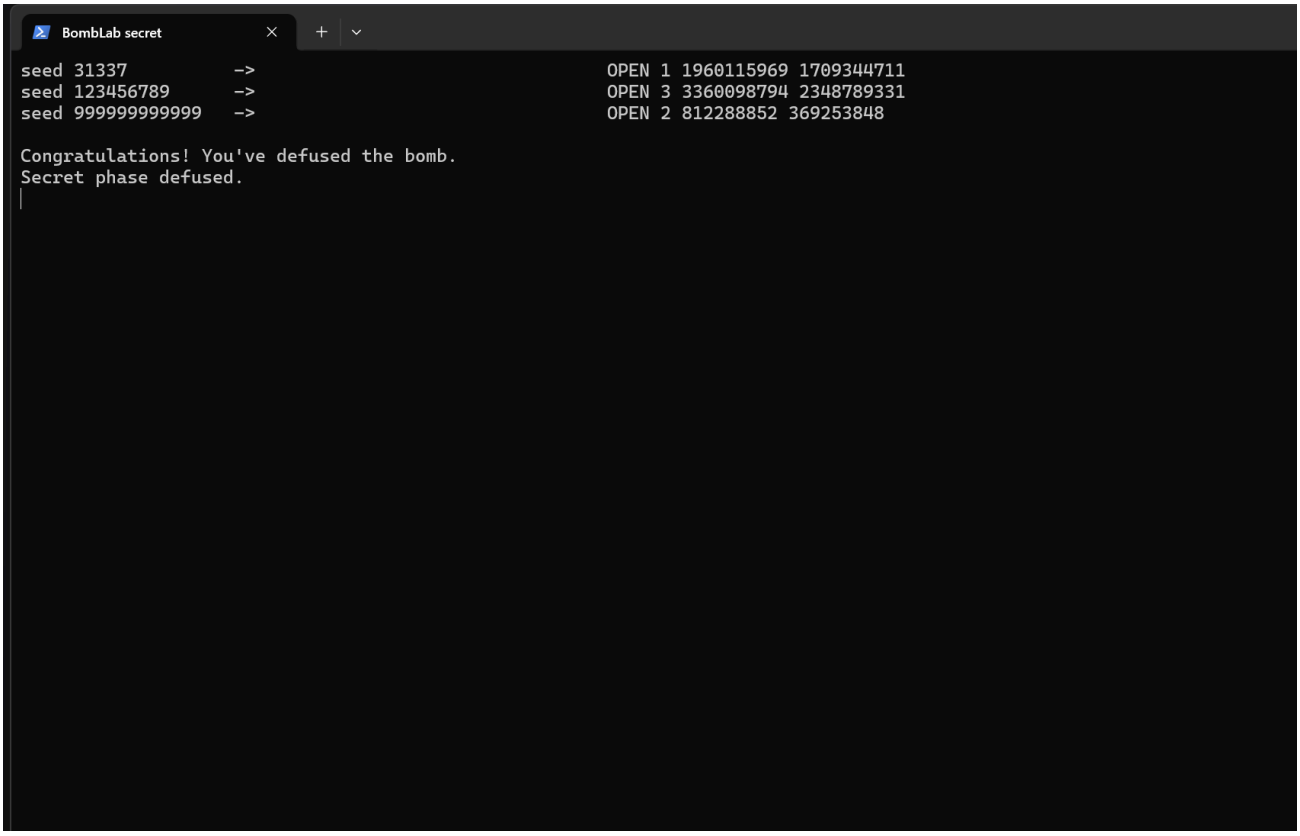
回归结果：

```
Public tests: 60/60
Secret public tests: 10/10
```

9. 运行结果截图


BombLab public_test
Phase 5 seed 123: PASS
Phase 5 seed 2026: PASS
Phase 5 seed 65535: PASS
Phase 5 seed 987654321: PASS
Phase 5 seed 18446744073709551615: PASS
Phase 6 seed 0: PASS
Phase 6 seed 1: PASS
Phase 6 seed 7: PASS
Phase 6 seed 42: PASS
Phase 6 seed 99: PASS
Phase 6 seed 123: PASS
Phase 6 seed 2026: PASS
Phase 6 seed 65535: PASS
Phase 6 seed 987654321: PASS
Phase 6 seed 18446744073709551615: PASS
Public tests: 60/60
Secret seed 0: PASS
Secret seed 1: PASS
Secret seed 7: PASS
Secret seed 42: PASS
Secret seed 99: PASS
Secret seed 123: PASS
Secret seed 2026: PASS
Secret seed 65535: PASS
Secret seed 987654321: PASS
Secret seed 18446744073709551615: PASS
Secret public tests: 10/10
=== all done ===


BombLab defuse
Phase 1 challenge: 0x29daa34ba1c52d21
Phase 1 defused.
Phase 2 challenge: 0x202e7457e0f6f34d
Phase 2 defused.
Phase 3 challenge: 0xf8745a73ea7b215f
Phase 3 defused.
Phase 4 challenge: 0xcf3acbb0e7ed57fb
Phase 4 defused.
Phase 5 challenge: 0xf6b3429e48af81a2
Phase 5 defused.
Phase 6 challenge: 0xd2b28b85aaa63ee6
Phase 6 defused.
Congratulations! You've defused the bomb.
Secret phase defused.



10. 工具与 AI 辅助使用说明

10.1 使用的工具

工具	用途
`objdump -d bomb`	反汇编，逐关读 `phase_0N` 及其 helper 的控制流与立即数
`objdump -s -j .rodata`	导出 `p5_heads/p5_nodes/p6_ops` 与第 3 关跳转表等静态数据
gdb	在 `secret_gate`(0x402700)、`secret_phase` 下断点，`info registers` 读运行时 challenge/context；用 `x/` 查看节点表
`./bomb --seed N --phase K`	单关动态验证，喂候选答案看是否 `defused`
`python3 public_test.py`	60 个公开用例 + 10 个 Honor 用例回归
WSL Ubuntu + gcc	编译与运行

调试期间只使用 gdb 的观察能力（`break`、`info registers`、`x/`、`set pagination off`），没有用 `set` 修改炸弹的寄存器或内存来绕过任何检查。

10.2 AI 辅助范围

本次作业使用了 AI 助手（DeepSeek Harness）协助分析汇编。

AI 负责的部分是阅读 objdump 反汇编，把每关从 challenge 推导期望值的控制流、寄存器数据流和立即数含义讲清楚，并协助排查 Honor 关的格式问题。

这些分析过程 AI 都逐条讲解过，本人对照汇编指令和算式做了复核，能够独立解释每一关的计算关系；defuser.c 中每个求解函数在提交前都用公开 seed 与额外若干 seed 实际验证，确认输出 defused 与 Secret phase defused。

报告中的全部数值均来自真实运行，没有编造。

10.3 参考资料

- 课程主页 https://ics-26fall-fdu.github.io/：Lab2 BombLab 实验文档；本仓库 README.md 与 AGENTS.md（实验要求与 AI 使用边界）。
- Intel 64 and IA-32 Architectures Software Developer's Manual, Volume 2（指令集参考）：imul、shr、shl、xor、lea、sete、jmp *table(,%rax,8) 的语义与操作数编码。
- Bryant & O'Hallaron, Computer Systems: A Programmer's Perspective, 3rd ed., 第 3 章「程序的机器级表示」：x86-64 寄存器约定、栈帧与参数传递、递归、switch 跳转表、结构体对齐与函数指针。
- man objdump、man gdb，GDB 官方文档中 info registers、x/ 命令的格式说明。